

```

# -*- coding: utf-8 -*-
"""DQN_Latest.ipynb

Automatically generated by Colab.

Original file is located at
https://colab.research.google.com/drive/17va1mWCMdlxqFQkc-96HYKU8tagY0UTc
"""

!pip install gymnasium torch matplotlib numpy

import io
import json
import math
import os
import random
import zipfile
import numpy as np
import gymnasium as gym
from gymnasium import spaces

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.patches as mpatches
from matplotlib.colors import Normalize
from matplotlib.cm import ScalarMappable

# =====
# Constants
# =====
ROOM_W = 10.0
ROOM_H = 10.0
DT = 0.1
MAX_STEPS = 500
MIN_SPAWN_DIST = 5.0

GOAL_RADIUS = 0.4
COLLISION_DIST = 0.3
DA_MIN = 0.8
WALL_MARGIN = 1.0
WALL_PENALTY = -1.5

K_ANGULAR = 1.0
K_PENALTY = 2.0
SUCCESS_REWARD = 10.0
COLLISION_PENALTY = -10.0

DISCRETE_ACTIONS = {
    0: -1.50, # Counterclockwise (fast)
    1: -0.75, # Counterclockwise (slow)
    2: 0.00, # Straight
    3: 0.75, # Clockwise (slow)
    4: 1.50, # Clockwise (fast)
}
LINEAR_VELOCITY = 0.5

# =====
# Dark theme palette (shared by all render helpers)
# =====
_BG = "#0d1117"
_PANEL = "#161b22"
_BORDER = "#30363d"
_MUTED = "#8b949e"
_REWARD_CMAP = "RdYlGn"
_REWARD_NORM = Normalize(vmin=-6, vmax=12)

# =====
# Helper classes
# =====
class Person:
    """A moving pedestrian."""
    def __init__(self, x, y, speed=0.3):
        self.x = x
        self.y = y
        self.speed = speed
        self.angle = random.uniform(0, 2 * math.pi)

    def step(self, dt, room_w, room_h):

```

```

        self.x += self.speed * math.cos(self.angle) * dt
        self.y += self.speed * math.sin(self.angle) * dt
        if self.x < 0.5 or self.x > room_w - 0.5:
            self.angle = math.pi - self.angle
        if self.y < 0.5 or self.y > room_h - 0.5:
            self.angle = -self.angle
        self.x = np.clip(self.x, 0.5, room_w - 0.5)
        self.y = np.clip(self.y, 0.5, room_h - 0.5)

class StaticObstacle:
    """A fixed obstacle."""
    def __init__(self, x, y, radius=0.4):
        self.x = x
        self.y = y
        self.radius = radius

# =====
# Main Environment
# =====

class WheelchairNavEnv(gym.Env):
    """
    Wheelchair navigation environment with built-in matplotlib rendering.

    render_mode options
    -----
    None          - no rendering (fastest, use for training)
    "rgb_array"   - render() returns an HxWx3 uint8 numpy array
    "png_bytes"   - render() returns raw PNG bytes (for Colab widgets)
    "save_png"    - render() saves a .png file to disk and returns the path
    """
    metadata = {"render_modes": ["human", "rgb_array", "png_bytes", "save_png"]}

    def __init__(
        self,
        action_type = "discrete",
        n_people    = 3,
        n_obstacles = 4,
        render_mode = None,
        max_steps   = MAX_STEPS,
        seed        = None,
    ):
        super().__init__()

        self.action_type = action_type
        self.n_people    = n_people
        self.n_obstacles = n_obstacles
        self.render_mode = render_mode
        self.max_steps   = max_steps
        self._rng        = np.random.default_rng(seed)

        self.agent_x = 0.0
        self.agent_y = 0.0
        self.agent_yaw = 0.0
        self.goal_x = 0.0
        self.goal_y = 0.0
        self.people: list[Person] = []
        self.obstacles: list[StaticObstacle] = []
        self._step_count = 0
        self._initial_dist = 0.0
        self._diag = math.hypot(ROOM_W, ROOM_H)

        # Trajectory buffer - filled each episode for the map trail
        self._traj_x: list[float] = []
        self._traj_y: list[float] = []
        self._traj_r: list[float] = []  # reward at each step (for colour)

        # Observation space: 7 normalised floats in [-1, 1]
        self.observation_space = spaces.Box(
            low = np.array([-1, 0, 0, -1, 0, 0, -1], dtype=np.float32),
            high = np.array([ 1, 1, 1,  1, 1, 1,  1], dtype=np.float32),
        )

        # Action space
        if action_type == "discrete":
            self.action_space = spaces.Discrete(5)
        else:
            self.action_space = spaces.Box(
                low = np.array([-1.5], dtype=np.float32),
                high = np.array([ 1.5], dtype=np.float32),
            )

```

```

# -----
# Core API
# -----
def reset(self, *, seed=None, options=None):
    super().reset(seed=seed)
    if seed is not None:
        self._rng = np.random.default_rng(seed)

    # Agent spawn
    self.agent_x = float(self._rng.uniform(1.0, ROOM_W - 1.0))
    self.agent_y = float(self._rng.uniform(1.0, ROOM_H - 1.0))
    self.agent_yaw = float(self._rng.uniform(-math.pi, math.pi))

    # Goal spawn (must be >= MIN_SPAWN_DIST away)
    while True:
        gx = float(self._rng.uniform(1.0, ROOM_W - 1.0))
        gy = float(self._rng.uniform(1.0, ROOM_H - 1.0))
        if math.hypot(gx - self.agent_x, gy - self.agent_y) >= MIN_SPAWN_DIST:
            self.goal_x, self.goal_y = gx, gy
            break

    # Static obstacles
    self.obstacles = []
    for _ in range(self.n_obstacles):
        for _ in range(50):
            ox = float(self._rng.uniform(0.8, ROOM_W - 0.8))
            oy = float(self._rng.uniform(0.8, ROOM_H - 0.8))
            if (math.hypot(ox - self.agent_x, oy - self.agent_y) > 1.2 and
                math.hypot(ox - self.goal_x, oy - self.goal_y) > 1.2):
                self.obstacles.append(StaticObstacle(ox, oy))
                break

    # Moving people
    self.people = []
    for _ in range(self.n_people):
        for _ in range(50):
            px = float(self._rng.uniform(1.0, ROOM_W - 1.0))
            py = float(self._rng.uniform(1.0, ROOM_H - 1.0))
            if math.hypot(px - self.agent_x, py - self.agent_y) > 1.5:
                self.people.append(Person(px, py))
                break

    self._step_count = 0
    self._initial_dist = math.hypot(
        self.goal_x - self.agent_x, self.goal_y - self.agent_y
    )

    # Clear trajectory
    self._traj_x = [self.agent_x]
    self._traj_y = [self.agent_y]
    self._traj_r = []

    return self._get_obs(), self._get_info()

def step(self, action):
    if self.action_type == "discrete":
        angular_vel = DISCRETE_ACTIONS[int(action)]
    else:
        angular_vel = float(np.clip(np.asarray(action).flat[0], -1.5, 1.5))

    self.agent_yaw += angular_vel * DT
    self.agent_yaw = (self.agent_yaw + math.pi) % (2 * math.pi) - math.pi
    self.agent_x += LINEAR_VELOCITY * math.cos(self.agent_yaw) * DT
    self.agent_y += LINEAR_VELOCITY * math.sin(self.agent_yaw) * DT
    self.agent_x = float(np.clip(self.agent_x, 0.3, ROOM_W - 0.3))
    self.agent_y = float(np.clip(self.agent_y, 0.3, ROOM_H - 0.3))

    for p in self.people:
        p.step(DT, ROOM_W, ROOM_H)

    self._step_count += 1
    reward, terminated = self._compute_reward()
    truncated = self._step_count >= self.max_steps

    # Append to trajectory
    self._traj_x.append(self.agent_x)
    self._traj_y.append(self.agent_y)
    self._traj_r.append(reward)

    return self._get_obs(), reward, terminated, truncated, self._get_info()

def render(self):

```

```

"""
Gymnasium render() - behaviour depends on render_mode set at init.

render_mode="rgb_array" → returns HxWx3 uint8 numpy array
render_mode="png_bytes" → returns raw PNG bytes (Colab widget ready)
render_mode="save_png" → saves wheelchair_render.png, returns path
render_mode=None → returns None
"""

if self.render_mode is None:
    return None
png = self._render_map_png()
if self.render_mode == "png_bytes":
    return png
if self.render_mode == "save_png":
    path = "wheelchair_render.png"
    with open(path, "wb") as f:
        f.write(png)
    return path
if self.render_mode == "rgb_array":
    import PIL.Image
    img = PIL.Image.open(io.BytesIO(png))
    return np.array(img.convert("RGB"))
return None

def render_frame(self, obs=None, q_vals=None, action_idx=None,
                 episode=None, step=None):
    """
    Four-panel frame: map + reward breakdown + observation vector + Q-values.
    Returns PNG bytes. Pass q_vals and action_idx from your agent for the
    Q-value panel; if omitted those panels show placeholder text.

    Designed to be called from sarsa_HP.py like:
    png = env.render_frame(obs=obs, q_vals=q_vals, action_idx=action)
    """
    return self._render_four_panel(obs, q_vals, action_idx, episode, step)

def close(self):
    plt.close("all")

# -----
# Reward
# -----
def _compute_reward(self):
    terminated = False
    dc = self._dist_to_goal()
    dg = self._initial_dist

    ratio = dc / dg if dg > 0 else 1.0
    rd = 2 ** ratio
    R_d = rd if dc < dg else max(1.0, rd)

    theta = self._angle_to_goal()
    r_theta = K_ANGULAR * (1 - math.cos(theta))
    R_theta = r_theta if abs(theta) < math.pi / 2 else -r_theta

    hd = self._count_nearby_people()
    da = self._min_dist_to_any()
    if hd > 0 and da < DA_MIN:
        P = hd * (-K_PENALTY)
    elif hd == 0 and da < DA_MIN:
        P = -K_PENALTY
    else:
        P = 0.0
    R_a = P

    dw = self._dist_to_wall()
    R_c = 0.0
    if da < COLLISION_DIST or dw < COLLISION_DIST:
        R_c = COLLISION_PENALTY
        terminated = True

    R_s = 0.0
    if dc < GOAL_RADIUS:
        R_s = SUCCESS_REWARD
        terminated = True

    R_w = WALL_PENALTY if dw < WALL_MARGIN else 0.0
    R_eff = -0.01
    total = (R_d * R_theta) + R_a + R_c + R_s + R_eff + R_w
    return float(total), terminated

def reward_breakdown(self) -> dict:
    """

```

```

Return a dict of every reward component at the current state.
Useful for the reward-bars panel and for debugging.
"""
dc    = self._dist_to_goal()
dg    = self._initial_dist
ratio = dc / dg if dg > 0 else 1.0
rd    = 2 ** ratio
R_d   = rd if dc < dg else max(1.0, rd)
theta = self._angle_to_goal()
r_th  = K_ANGULAR * (1 - math.cos(theta))
R_th  = r_th if abs(theta) < math.pi / 2 else -r_th
R_p   = R_d * R_th
hd    = self._count_nearby_people()
da    = self._min_dist_to_any()
dw    = self._dist_to_wall()
if hd > 0 and da < DA_MIN:
    R_a = hd * (-K_PENALTY)
elif hd == 0 and da < DA_MIN:
    R_a = -K_PENALTY
else:
    R_a = 0.0
R_w   = WALL_PENALTY if dw < WALL_MARGIN else 0.0
R_c   = COLLISION_PENALTY if (da < COLLISION_DIST or dw < COLLISION_DIST) else 0.0
R_s   = SUCCESS_REWARD if dc < GOAL_RADIUS else 0.0
R_eff = -0.01
return {
    "R_d (dist rate)": round(R_d, 4),
    "R_th (angular)": round(R_th, 4),
    "R_prog (Rd×Rth)": round(R_p, 4),
    "R_a (proximity)": round(R_a, 4),
    "R_w (wall pen.)": round(R_w, 4),
    "R_c (collision)": round(R_c, 4),
    "R_s (success)": round(R_s, 4),
    "R_eff (step pen.)": round(R_eff, 4),
    "TOTAL": round(R_p + R_a + R_w + R_c + R_s + R_eff, 4),
}

# -----
# Observation / info
# -----
def _get_obs(self) -> np.ndarray:
    theta_goal = self._angle_to_goal() / math.pi
    dist_goal  = self._dist_to_goal() / self._diag
    r_min      = self._min_dist_to_obstacle() / self._diag
    theta_obs  = self._angle_to_nearest_obstacle() / math.pi
    hd         = self._count_nearby_people() / max(1, self.n_people)
    collision   = 1.0 if self._min_dist_to_any() < COLLISION_DIST else 0.0
    yaw_norm   = self.agent_yaw / math.pi
    obs = np.array(
        [theta_goal, dist_goal, r_min, theta_obs, hd, collision, yaw_norm],
        dtype=np.float32,
    )
    return np.clip(obs, -1.0, 1.0)

def _get_info(self) -> dict:
    return {
        "agent_pos": (round(self.agent_x, 3), round(self.agent_y, 3)),
        "agent_yaw": round(math.degrees(self.agent_yaw), 2),
        "goal_pos": (round(self.goal_x, 3), round(self.goal_y, 3)),
        "dist_to_goal": round(self._dist_to_goal(), 3),
        "step": self._step_count,
    }

# -----
# Distance / angle helpers
# -----
def _dist_to_goal(self) -> float:
    return math.hypot(self.goal_x - self.agent_x, self.goal_y - self.agent_y)

def _angle_to_goal(self) -> float:
    raw = math.atan2(self.goal_y - self.agent_y, self.goal_x - self.agent_x)
    diff = raw - self.agent_yaw
    return (diff + math.pi) % (2 * math.pi) - math.pi

def _min_dist_to_obstacle(self) -> float:
    if not self.obstacles:
        return self._diag
    return min(
        math.hypot(o.x - self.agent_x, o.y - self.agent_y) - o.radius
        for o in self.obstacles
    )

```

```

def _angle_to_nearest_obstacle(self) -> float:
    if not self.obstacles:
        return 0.0
    nearest = min(self.obstacles,
                  key=lambda o: math.hypot(o.x - self.agent_x, o.y - self.agent_y))
    raw = math.atan2(nearest.y - self.agent_y, nearest.x - self.agent_x)
    diff = raw - self.agent_yaw
    return (diff + math.pi) % (2 * math.pi) - math.pi

def _min_dist_to_any(self) -> float:
    dists = []
    for o in self.obstacles:
        dists.append(math.hypot(o.x - self.agent_x, o.y - self.agent_y) - o.radius)
    for p in self.people:
        dists.append(math.hypot(p.x - self.agent_x, p.y - self.agent_y))
    return min(dists) if dists else self._diag

def _count_nearby_people(self) -> int:
    return sum(
        1 for p in self.people
        if math.hypot(p.x - self.agent_x, p.y - self.agent_y) < DA_MIN
    )

def _dist_to_wall(self) -> float:
    return min(
        self.agent_x,
        ROOM_W - self.agent_x,
        self.agent_y,
        ROOM_H - self.agent_y,
    )

# -----
# Internal rendering helpers
# -----
@staticmethod
def _fig_to_png(fig) -> bytes:
    buf = io.BytesIO()
    fig.savefig(buf, format="png", dpi=110,
               facecolor=fig.get_facecolor(), bbox_inches="tight")
    buf.seek(0)
    data = buf.read()
    buf.close()
    plt.close(fig)
    return data

@staticmethod
def _style_axes(axes):
    """Apply dark theme to a list (or single) of axes."""
    if not hasattr(axes, "__iter__"):
        axes = [axes]
    for ax in axes:
        ax.set_facecolor(_PANEL)
        for sp in ax.spines.values():
            sp.set_edgecolor(_BORDER)
        ax.tick_params(colors=_MUTED, labelsize=8)

@staticmethod
def _reward_colour(r: float):
    return plt.get_cmap(_REWARD_CMAP)(_REWARD_NORM(float(np.clip(r, -6, 12))))

# — Panel: top-down map —————
def _draw_map(self, ax):
    ax.clear()
    ax.set_facecolor(_BG)
    ax.set_xlim(-0.4, ROOM_W + 0.4)
    ax.set_ylim(-0.4, ROOM_H + 0.4)
    ax.set_aspect("equal")
    ax.tick_params(colors=_MUTED, labelsize=8)
    for sp in ax.spines.values():
        sp.set_edgecolor("#333")

    # Grid
    for v in range(0, int(ROOM_W) + 1, 2):
        ax.axvline(v, color="#1c1c2e", lw=0.5)
    for h in range(0, int(ROOM_H) + 1, 2):
        ax.axhline(h, color="#1c1c2e", lw=0.5)

    # Room boundary
    ax.add_patch(mpatches.Rectangle(
        (0, 0), ROOM_W, ROOM_H,
        lw=2, edgecolor="#555577", facecolor="none"))

    # Trajectory coloured by reward

```

```

tx, ty, tr = self._traj_x, self._traj_y, self._traj_r
if len(tx) > 1:
    for i in range(len(tx) - 1):
        r_val = tr[i] if i < len(tr) else 0.0
        ax.plot([tx[i], tx[i+1]], [ty[i], ty[i+1]],
                color=self._reward_colour(r_val),
                lw=1.5, alpha=0.75)

# Static obstacles
for ob in self.obstacles:
    ax.add_patch(plt.Circle((ob.x, ob.y), ob.radius,
                           color="#7B4F2E", alpha=0.9, zorder=3))
    ax.text(ob.x, ob.y, "O", ha="center", va="center",
           color="white", fontsize=7, fontweight="bold", zorder=4)

# Moving people
for i, p in enumerate(self.people):
    ax.add_patch(plt.Circle((p.x, p.y), DA_MIN,
                           color="#FF8C00", alpha=0.07))
    ax.add_patch(plt.Circle((p.x, p.y), DA_MIN, fill=False,
                           edgecolor="#FFA500", lw=0.8, ls="--", alpha=0.4))
    ax.add_patch(plt.Circle((p.x, p.y), 0.22,
                           color="#FF6600", zorder=5))
    ax.text(p.x, p.y + 0.42, f"P{i+1}", ha="center",
           fontsize=7, color="#FFA500", fontweight="bold", zorder=6)

# Goal
ax.add_patch(plt.Circle((self.goal_x, self.goal_y), GOAL_RADIUS,
                       color="#00ff99", alpha=0.15))
ax.add_patch(plt.Circle((self.goal_x, self.goal_y), GOAL_RADIUS,
                       fill=False, edgecolor="#00ff99", lw=1.8))
ax.plot(self.goal_x, self.goal_y, "*",
        color="#00ff99", ms=15, mec="white", mew=0.5, zorder=6)
ax.text(self.goal_x, self.goal_y + 0.62, "GOAL", ha="center",
        fontsize=8, color="#00ff99", fontweight="bold", zorder=7)

# Dashed line agent → goal
ax.plot([self.agent_x, self.goal_x], [self.agent_y, self.goal_y],
        "--", color="#00ff99", alpha=0.2, lw=0.8)

# Agent
da = self._min_dist_to_any()
a_col = ("#ff2222" if da < COLLISION_DIST else
        "#ffaa00" if da < DA_MIN else "#3399ff")
ax.add_patch(plt.Circle((self.agent_x, self.agent_y), 0.28,
                       color=a_col, zorder=7))

ax.annotate(
    "",
    xy=(self.agent_x + 0.60 * math.cos(self.agent_yaw),
        self.agent_y + 0.60 * math.sin(self.agent_yaw)),
    xytext=(self.agent_x, self.agent_y),
    arrowprops=dict(arrowstyle="→", color="white", lw=2.0),
    zorder=8,
)

# Reward colourbar
sm = ScalarMappable(cmap=_REWARD_CMAP, norm=_REWARD_NORM)
sm.set_array([])
cbar = ax.get_figure().colorbar(
    sm, ax=ax, fraction=0.022, pad=0.01, shrink=0.5)
cbar.set_label("Step reward", color=_MUTED, fontsize=7)
cbar.ax.yaxis.set_tick_params(labelsize=6)
plt.setp(cbar.ax.yaxis.get_ticklabels(), color=_MUTED)

# Legend
ax.legend(handles=[
    mpatches.Patch(color="#3399ff", label="Agent"),
    mpatches.Patch(color="#ffaa00", label="Near obj"),
    mpatches.Patch(color="#ff2222", label="Collision"),
    mpatches.Patch(color="#00ff99", label="Goal"),
    mpatches.Patch(color="#FF6600", label="Person"),
    mpatches.Patch(color="#7B4F2E", label="Obstacle")
], loc="upper right", fontsize=6.5, framealpha=0.5,
facecolor=_BG, labelcolor="white", edgecolor="#444")

dc = self._dist_to_goal()
ang = math.degrees(self._angle_to_goal())
st = ("COLLISION" if da < COLLISION_DIST else
      "NEAR" if da < DA_MIN else "CLEAR")
ax.set_title(f"Dist {dc:.2f}m Angle {ang:+.0f}° [{st}]",
            color="white", fontsize=9, pad=4)
ax.set_xlabel("x (m)", color=_MUTED, fontsize=8)
ax.set_ylabel("y (m)", color=_MUTED, fontsize=8)

```

```

# — Panel: reward breakdown bars —————
def _draw_rewardBars(self, ax):
    ax.clear()
    ax.set_facecolor(_BG)
    for sp in ax.spines.values():
        sp.set_edgecolor("#333")

    bd = self.reward_breakdown()
    labels = list(bd.keys())
    vals = list(bd.values())
    cols = [
        "#3a9de0" if l == "TOTAL" else
        "#2ecc71" if v > 0.05 else
        "#e74c3c" if v < -0.05 else "#555"
        for l, v in zip(labels, vals)
    ]
    yp = list(range(len(labels)))
    bars = ax.barh(yp, vals, color=cols, edgecolor="#111", lw=0.4)
    ax.set_yticks(yp)
    ax.set_yticklabels(labels, color="white", fontsize=8)
    ax.axvline(0, color="#555", lw=0.8)
    ax.tick_params(axis="x", colors=_MUTED, labelsize=7)
    ax.set_title("Reward Components", color="white", fontsize=9, pad=4)
    ax.invert_yaxis()
    for bar, v in zip(bars, vals):
        ax.text(
            v + (0.05 if v >= 0 else -0.05),
            bar.get_y() + bar.get_height() / 2,
            f"{v:+.3f}", va="center",
            ha="left" if v >= 0 else "right",
            fontsize=7.5, color="white",
        )

# — Panel: observation vector bars —————
def _draw_obsBars(self, ax, obs):
    ax.clear()
    ax.set_facecolor(_BG)
    for sp in ax.spines.values():
        sp.set_edgecolor("#333")

    labels = [
        "obs[0] angle/goal",
        "obs[1] dist/goal",
        "obs[2] min dist",
        "obs[3] angle/obs",
        "obs[4] people",
        "obs[5] collision",
        "obs[6] yaw",
    ]
    cols = [
        "#e74c3c" if i == 5 and v > 0.5 else
        "#2ecc71" if i == 5 else
        "#e67e22" if abs(v) > 0.7 else "#7f8c8d"
        for i, v in enumerate(obs)
    ]
    yp = list(range(len(obs)))
    ax.barh(yp, obs, color=cols, edgecolor="#111", lw=0.4)
    ax.set_yticks(yp)
    ax.set_yticklabels(labels, color="white", fontsize=7.5)
    ax.axvline(0, color="#555", lw=0.8)
    ax.set_xlim(-1.15, 1.15)
    ax.tick_params(axis="x", colors=_MUTED, labelsize=7)
    ax.set_title("Observation Vector", color="white", fontsize=9, pad=4)
    ax.invert_yaxis()
    for i, v in enumerate(obs):
        ax.text(
            v + (0.04 if v >= 0 else -0.04), i,
            f"{v:+.2f}", va="center",
            ha="left" if v >= 0 else "right",
            fontsize=7, color="white",
        )

# — Panel: Q-value bars —————
@staticmethod
def _draw_qBars(ax, q_vals, action_idx):
    ax.clear()
    ax.set_facecolor(_BG)
    for sp in ax.spines.values():
        sp.set_edgecolor("#333")

    if q_vals is None:

```

```

        ax.text(0.5, 0.5, "Q-values not provided",
                ha="center", va="center", color=_MUTED,
                transform=ax.transAxes, fontsize=9)
    ax.set_title("Q-values per Action", color="white", fontsize=9, pad=4)
    return

    best = int(np.argmax(q_vals)) if action_idx is None else action_idx
    cols = ["#f0883e" if i == best else "#58a6ff"
            for i in range(len(q_vals))]
    xlabs = [f"A{i}: {DISCRETE_ACTIONS[i]:+.2f}" for i in range(len(q_vals))]
    bars = ax.bar(range(len(q_vals)), q_vals,
                  color=cols, edgecolor="#111", lw=0.4)
    ax.set_xticks(range(len(q_vals)))
    ax.set_xticklabels(xlabs, color="white", fontsize=7.5, rotation=10)
    ax.axhline(0, color="#555", lw=0.8)
    ax.tick_params(axis="y", colors=_MUTED, labels=7)
    ax.set_title("Q-values (orange = chosen)", color="white", fontsize=9, pad=4)
    for bar, v in zip(bars, q_vals):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            v + (0.03 if v >= 0 else -0.03),
            f"{v:+.2f}", ha="center",
            va="bottom" if v >= 0 else "top",
            fontsize=7, color="white",
        )

# — Four-panel episode frame —————
def _render_four_panel(self, obs=None, q_vals=None,
                       action_idx=None, episode=None, step=None) -> bytes:
    fig = plt.figure(figsize=(16, 6), facecolor=_BG)
    gs = gridspec.GridSpec(
        1, 4, figure=fig,
        left=0.05, right=0.97, top=0.88, bottom=0.10, wspace=0.42)
    axes = [fig.add_subplot(gs[0, i]) for i in range(4)]

    ep_str = f"Ep {episode}" if episode is not None else ""
    st_str = f" Step {step}" if step is not None else ""
    act_str = ""
    if action_idx is not None:
        act_str = f" | Action {action_idx} ({DISCRETE_ACTIONS[action_idx]:+.2f} rad/s)"
    q_str = ""
    if q_vals is not None:
        q_str = f" | Best Q {max(q_vals):+.2f}"

    fig.suptitle(
        f"SARSA Agent {ep_str}{st_str}{act_str}{q_str}",
        color="white", fontsize=10, fontweight="bold")

    self._draw_map(axes[0])
    self._draw_reward_bars(axes[1])

    if obs is not None:
        self._draw_obs_bars(axes[2], obs)
    else:
        axes[2].set_facecolor(_BG)
        axes[2].text(0.5, 0.5, "obs not provided",
                    ha="center", va="center", color=_MUTED,
                    transform=axes[2].transAxes, fontsize=9)

    self._draw_q_bars(axes[3], q_vals, action_idx)

    return self._fig_to_png(fig)

# — Simple map-only PNG —————
def _render_map_png(self) -> bytes:
    fig, ax = plt.subplots(figsize=(6, 6), facecolor=_BG)
    self._draw_map(ax)
    return self._fig_to_png(fig)

# =====
# Wheelchair Navigation – DQN Agent (Google Colab Ready)
# Based on: Afonso & Ferreira Jr. (2023), Research Square
# Environment: WheelchairNavEnv (user-supplied)
# 2000 episodes | Discrete action space (5 actions)
# =====

import warnings
import torch
import torch.nn as nn
import torch.optim as optim
from collections import deque

```

```
warnings.filterwarnings("ignore")

# — DQN Network —————
class DQNNetwork(nn.Module):
    """
    Three-layer FC Q-network.
    Input(7) -> 400 -> 300 -> 5 actions
    Matches the paper's network design (Section 3.3).
    """
    def __init__(self, state_dim=7, action_dim=5, h1=400, h2=300):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, h1), nn.ReLU(),
            nn.Linear(h1, h2), nn.ReLU(),
            nn.Linear(h2, action_dim),
        )

    def forward(self, x):
        return self.net(x)

# — Replay Buffer —————
class ReplayBuffer:
    def __init__(self, capacity=1_000_000):
        self.buf = deque(maxlen=capacity)

    def push(self, s, a, r, ns, done):
        self.buf.append((
            np.asarray(s, dtype=np.float32), int(a),
            float(r),
            np.asarray(ns, dtype=np.float32), float(done),
        ))

    def sample(self, batch_size):
        batch = random.sample(self.buf, batch_size)
        s, a, r, ns, d = zip(*batch)
        return (
            torch.FloatTensor(np.stack(s)),
            torch.LongTensor(a),
            torch.FloatTensor(r),
            torch.FloatTensor(np.stack(ns)),
            torch.FloatTensor(d),
        )

    def __len__(self):
        return len(self.buf)

# — DQN Agent —————
class DQNAgent:
    """
    DQN with epsilon-greedy, hard target-network updates (every TAU steps),
    Adam optimiser, gradient clipping.
    Hyperparameters: Table 3 of the paper.
    """
    GAMMA = 0.99
    LR = 0.00025
    BATCH_SIZE = 64
    BUFFER_SIZE = 1_000_000
    TAU = 2_000 # hard target-update frequency (global steps)
    EPSILON_START = 1.0
    EPSILON_DECAY = 0.99 # per-episode multiplier
    EPSILON_MIN = 0.05
    TRAIN_START = 64

    def __init__(self, state_dim=7, action_dim=5, device=None):
        self.action_dim = action_dim
        self.device = device or (
            torch.device("cuda") if torch.cuda.is_available()
            else torch.device("cpu")
        )
        print(f"[DQN] Device: {self.device}")

        self.online = DQNNetwork(state_dim, action_dim).to(self.device)
        self.target = DQNNetwork(state_dim, action_dim).to(self.device)
        self.target.load_state_dict(self.online.state_dict())
        self.target.eval()

        self.optimiser = optim.Adam(self.online.parameters(), lr=self.LR)
        self.loss_fn = nn.MSELoss()
        self.buffer = ReplayBuffer(self.BUFFER_SIZE)
        self.ensilon = self.EPSILON_START
```

```

self.step_count = 0 # cumulative environment steps

def select_action(self, state, greedy=False):
    if not greedy and random.random() < self.epsilon:
        return random.randint(0, self.action_dim - 1)
    s = torch.FloatTensor(state).unsqueeze(0).to(self.device)
    with torch.no_grad():
        return int(self.online(s).argmax(1).item())

def q_values(self, state):
    s = torch.FloatTensor(state).unsqueeze(0).to(self.device)
    with torch.no_grad():
        return self.online(s).squeeze(0).cpu().numpy()

def learn(self):
    if len(self.buffer) < self.TRAIN_START:
        return None
    s, a, r, ns, d = [t.to(self.device)
                       for t in self.buffer.sample(self.BATCH_SIZE)]
    q_curr = self.online(s).gather(1, a.unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        q_next = self.target(ns).max(1).values
        q_targ = r + self.GAMMA * q_next * (1.0 - d)
    loss = self.loss_fn(q_curr, q_targ)
    self.optimiser.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self.online.parameters(), 10.0)
    self.optimiser.step()
    self.step_count += 1
    if self.step_count % self.TAU == 0:
        self.target.load_state_dict(self.online.state_dict())
    return loss.item()

def end_episode(self):
    self.epsilon = max(self.EPSILON_MIN, self.epsilon * self.EPSILON_DECAY)

# — Training Loop —
def train_dqn(
    n_episodes = 2_000,
    max_steps = 500,
    n_people = 3,
    n_obstacles = 4,
    seed = 42,
    print_every = 50,
):
    """
    Train DQN for n_episodes. Returns (metrics dict, trained agent).

    Sample efficiency is measured as cumulative environment steps to
    reach convergence (rolling avg reward >= 75th percentile of all
    rolling averages) – comparable to:
        TD3: 243,332 steps
        SARSA: 400,914 steps
        DDPG: 333,293 steps
        DQN: <computed here>
    """
    env = WheelchairNavEnv(
        action_type = "discrete",
        n_people = n_people,
        n_obstacles = n_obstacles,
        render_mode = None,
        max_steps = max_steps,
        seed = seed,
    )
    agent = DQNAgent(state_dim=7, action_dim=5)

    # Per-episode logs
    ep_total_rewards = []
    ep_steps = []
    ep_success = []
    ep_collision = []
    ep_losses = []
    ep_epsilon = []
    ep_cumul_steps = [] # cumulative env steps after each episode

    WINDOW = 100
    cumul_steps = 0

    print(f"\n{' '*68}")
    print(f" DQN Training | {n_episodes} episodes | seed={seed} "
          f"| device={agent.device}")

```

```

print(T = "\n")

for ep in range(1, n_episodes + 1):
    obs, _ = env.reset(seed=seed + ep)
    total_r = 0.0
    losses = []
    success = False
    collision = False

    for step in range(max_steps):
        action = agent.select_action(obs)
        next_obs, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

        agent.buffer.push(obs, action, reward, next_obs, float(done))
        loss = agent.learn()
        if loss is not None:
            losses.append(loss)

        total_r += reward
        cumul_steps += 1
        obs = next_obs

        if terminated:
            if info.get("dist_to_goal", 999) < 0.4:
                success = True
            else:
                collision = True
            break

    agent.end_episode()

    ep_total_rewards.append(total_r)
    ep_steps.append(step + 1)
    ep_success.append(success)
    ep_collision.append(collision)
    ep_losses.append(float(np.mean(losses)) if losses else 0.0)
    ep_epsilon.append(agent.epsilon)
    ep_cumul_steps.append(cumul_steps)

    if ep % print_every == 0 or ep == 1:
        win = ep_total_rewards[-WINDOW:]
        avg_r = np.mean(win)
        sr = np.mean(ep_success[-WINDOW:]) * 100
        cr = np.mean(ep_collision[-WINDOW:]) * 100
        print(
            f"Ep {ep:>4}/{n_episodes} "
            f"AvgR(w{WINDOW})={avg_r:>8.2f} "
            f"TotR={total_r:>8.2f} "
            f"SR={sr:>5.1f}% CR={cr:>5.1f}% "
            f"eps={agent.epsilon:.4f} "
            f"CumulSteps={cumul_steps:},"
        )

env.close()

# — Rolling series —————
rewards_arr = np.array(ep_total_rewards)
success_arr = np.array(ep_success, dtype=float)
collision_arr = np.array(ep_collision, dtype=float)
cumul_arr = np.array(ep_cumul_steps)

def rolling(arr, w=WINDOW):
    return np.convolve(arr, np.ones(w)/w, mode='valid')

avg_rewards = rolling(rewards_arr)
sr_rolling = rolling(success_arr) * 100
cr_rolling = rolling(collision_arr) * 100

# — Convergence: first rolling-window index where avg >= 75th pct —
threshold = np.percentile(avg_rewards, 75)
converge_idx = next(
    (i for i, v in enumerate(avg_rewards) if v >= threshold), len(avg_rewards) - 1
)
# Episode number and cumulative steps AT convergence
converge_ep = converge_idx + WINDOW
converge_steps = int(cumul_arr[converge_ep - 1])

# — Sample efficiency (steps-based, matching TD3/SARSA/DDPG format) —
sample_eff_steps = converge_steps

# Also record episode where 50% rolling SR is first hit
samp_eff_ep = next(

```

```

    (i + WINDOW for i, v in enumerate(sr_rolling) if v >= 50.0),
    n_episodes,
)

# — Convergence Rate (%) —————
convergence_rate_pct = (converge_ep / n_episodes) * 100.0
post_convergence_pct = 100.0 - convergence_rate_pct

metrics = {
    # raw
    "ep_total_rewards" : ep_total_rewards,
    "ep_steps" : ep_steps,
    "ep_success" : ep_success,
    "ep_collision" : ep_collision,
    "ep_losses" : ep_losses,
    "ep_epsilon" : ep_epsilon,
    "ep_cumul_steps" : ep_cumul_steps,
    # rolling
    "avg_rewards" : avg_rewards,
    "sr_rolling" : sr_rolling,
    "cr_rolling" : cr_rolling,
    # summary
    "final_avg_reward" : float(avg_rewards[-1]),
    "final_success_rate" : float(sr_rolling[-1]),
    "convergence_ep" : converge_ep,
    "convergence_steps" : converge_steps,
    "convergence_rate_pct" : convergence_rate_pct,
    "post_convergence_pct" : post_convergence_pct,
    "sample_eff_steps" : sample_eff_steps,
    "samp_eff_ep_50pct" : samp_eff_ep,
    "total_successes" : int(sum(ep_success)),
    "total_collisions" : int(sum(ep_collision)),
    "total_steps" : cumul_steps,
    "window" : WINDOW,
    "n_episodes" : n_episodes,
    # Total reward summary
    "total_reward_sum" : float(np.sum(rewards_arr)),
    "avg_total_reward" : float(np.mean(rewards_arr)),
    "max_episode_reward" : float(np.max(rewards_arr)),
    "min_episode_reward" : float(np.min(rewards_arr)),
}

print(f"\n{' '*68}")
print("  TRAINING COMPLETE")
print(f"  Final avg reward (last {WINDOW} ep): {metrics['final_avg_reward']:+.3f}")
print(f"  Final success rate (last {WINDOW} ep): {metrics['final_success_rate']:.1f}%")
print(f"  Convergence episode: ep {converge_ep}")
print(f"  Convergence (cumulative steps): {converge_steps:,} steps")
print(f"")
print(f"  — Total Reward Summary —————")
print(f"    Sum of all episode rewards : {metrics['total_reward_sum']:.2f}")
print(f"    Average reward per episode : {metrics['avg_total_reward']:+.3f}")
print(f"    Best single episode reward : {metrics['max_episode_reward']:+.3f}")
print(f"    Worst single episode reward : {metrics['min_episode_reward']:+.3f}")
print(f"")
print(f"  — Convergence Rate —————")
print(f"    75th-pct threshold (avg reward): {threshold:+.3f}")
print(f"    Converged at episode : {converge_ep}")
print(f"    Converged at cumulative steps : {converge_steps:,}")
print(f"    Convergence rate : {convergence_rate_pct:.2f}% "
      f"(converged after {convergence_rate_pct:.1f}% of training)")
print(f"    Training remaining after conv. : {post_convergence_pct:.2f}%")
print(f"    Episodes to reach 50% SR : ep {samp_eff_ep}")
print(f"")
print(f"  — Sample Efficiency —————")
print(f"    DQN : {converge_steps:,} steps ← this run")
print(f"")
print(f"  Total successes / collisions: "
      f"{metrics['total_successes']} / {metrics['total_collisions']}")
print(f"  Total env steps: {cumul_steps:,}")
print(f"\n{' '*68}\n")

return metrics, agent

# — Plotting —————
def plot_metrics(metrics, save_path="dqn_metrics.png"):
    """
    Six-panel dark-theme figure:
    1. Total reward per episode + rolling avg
    2. Rolling average reward (shaded area)
    3. Convergence rate (avg reward vs 75th-pct threshold, x=cumul steps)
    4. DON sample efficiency (cumul steps curve, convergence point marked)

```

```

5. Success rate & collision rate (rolling %)
6. Steps per episode (rolling)
"""
_BG = "#0d1117"
_PANEL = "#161b22"
_MUTED = "#8b949e"

n = metrics["n_episodes"]
w = metrics["window"]
ep = np.arange(1, n + 1)
ep_r = np.arange(w, n + 1)

cumul = np.array(metrics["ep_cumul_steps"])
cumul_rol = cumul[w - 1:] # x-axis for rolling series (steps)

fig = plt.figure(figsize=(19, 11), facecolor=_BG)
fig.suptitle(
    f"DQN - Wheelchair Navigation | {n} Episodes "
    f"| Final SR: {metrics['final_success_rate']:.1f}% "
    f"| Sample Efficiency: {metrics['sample_eff_steps']:,} steps "
    f"(Converged ep {metrics['convergence_ep']})",
    color="white", fontsize=11, fontweight="bold", y=0.97,
)
gs = gridspec.GridSpec(
    2, 3, figure=fig,
    hspace=0.46, wspace=0.37,
    left=0.07, right=0.97,
    top=0.91, bottom=0.07,
)

def ax_at(r, c):
    ax = fig.add_subplot(gs[r, c])
    ax.set_facecolor(_PANEL)
    for sp in ax.spines.values():
        sp.set_edgecolor("#30363d")
    ax.tick_params(colors=_MUTED, labelsize=8)
    ax.xaxis.label.set_color(_MUTED)
    ax.yaxis.label.set_color(_MUTED)
    ax.title.set_color("white")
    return ax

def ksteps(x):
    """Format large step numbers as 'Nk'."""
    return [f"{int(v/1000)}k" if v >= 1000 else str(int(v)) for v in x]

# — 1. Total reward per episode —————
ax1 = ax_at(0, 0)
ax1.plot(ep, metrics["ep_total_rewards"],
         color="#58a6ff", lw=0.5, alpha=0.4, label="Per-episode")
ax1.plot(ep_r, metrics["avg_rewards"],
         color="#f0883e", lw=2.0, label=f"Rolling avg (w={w})")
ax1.axhline(0, color="#444", lw=0.8, ls="--")
ax1.set_title("Total Reward per Episode")
ax1.set_xlabel("Episode")
ax1.set_ylabel("Reward")
ax1.legend(fontsize=8, facecolor=_BG, labelcolor="white",
          edgecolor="#333", framealpha=0.6)

# — 2. Rolling average reward —————
ax2 = ax_at(0, 1)
ax2.plot(ep_r, metrics["avg_rewards"], color="#3fb950", lw=1.8)
ax2.fill_between(ep_r, metrics["avg_rewards"], alpha=0.18, color="#3fb950")
ax2.axhline(0, color="#444", lw=0.8, ls="--")
ax2.set_title(f"Rolling Average Reward (w={w})")
ax2.set_xlabel("Episode")
ax2.set_ylabel("Avg Reward")

# — 3. Convergence rate (x = cumulative steps) —————
threshold = np.percentile(metrics["avg_rewards"], 75)
ax3 = ax_at(0, 2)
ax3.plot(cumul_rol, metrics["avg_rewards"],
         color="#d2a8ff", lw=1.8, label="Avg reward")
ax3.axhline(threshold, color="#ff7b72", lw=1.5, ls="--",
            label=f"75th pct ({threshold:.2f})")
conv_s = metrics["convergence_steps"]
if metrics["convergence_ep"] < n:
    ax3.axvline(conv_s, color="#ffa657", lw=1.5, ls=":",
                label=f"Converged @ {conv_s:,} steps")
ax3.set_title("Convergence Rate (by Steps)")
ax3.set_xlabel("Cumulative Env Steps")
ax3.set_ylabel("Avg Reward")
xticks = ax3.get_xticks()
ax3.set_xticklabels(ksteps(xticks), fontsize=7)

```

```

ax3.set_xlabel('Steps (k)', fontsize=7)
ax3.legend(fontsize=7, facecolor=_BG, labelcolor="white",
           edgcolor="#333", framealpha=0.6)

# — 4. DQN sample efficiency (steps curve, no comparison bars) —
dqn_steps = metrics["sample_eff_steps"]
conv_ep    = metrics["convergence_ep"]
conv_pct   = metrics["convergence_rate_pct"]

ax4 = ax_at(1, 0)
ax4.plot(ep_r, cumul_roll / 1_000,
         color="#ffa657", lw=1.8, label="Cumul. steps (k)")
ax4.axvline(conv_ep, color="#56d364", lw=1.5, ls="--",
           label=f"Converged ep {conv_ep}")
ax4.scatter([conv_ep], [dqn_steps / 1_000],
           color="#56d364", s=60, zorder=5)
ax4.text(
    conv_ep + (n * 0.01), dqn_steps / 1_000 + 5,
    f"{dqn_steps:,} steps\n({conv_pct:.1f}% of training)",
    color="#56d364", fontsize=7.5, va="bottom",
)
ax4.set_title("DQN Sample Efficiency\n(Steps to Convergence)")
ax4.set_xlabel("Episode")
ax4.set_ylabel("Cumulative Env Steps (k)")
ax4.legend(fontsize=7, facecolor=_BG, labelcolor="white",
           edgcolor="#333", framealpha=0.6)

# — 5. Success rate & collision rate —
ax5 = ax_at(1, 1)
ax5.plot(ep_r, metrics["sr_rolling"],
         color="#56d364", lw=1.8, label="Success Rate")
ax5.plot(ep_r, metrics["cr_rolling"],
         color="#ff7b72", lw=1.8, label="Collision Rate")
ax5.set_ylim(0, 105)
ax5.set_title("Success Rate & Collision Rate")
ax5.set_xlabel("Episode")
ax5.set_ylabel("Rate (%)")
ax5.legend(fontsize=8, facecolor=_BG, labelcolor="white",
           edgcolor="#333", framealpha=0.6)
ax5.text(ep_r[-1], min(metrics["sr_rolling"][-1] + 3, 102),
        f"{metrics['sr_rolling'][-1]:.1f}%",
        color="#56d364", fontsize=8, ha="right")
ax5.text(ep_r[-1], min(metrics["cr_rolling"][-1] + 3, 102),
        f"{metrics['cr_rolling'][-1]:.1f}%",
        color="#ff7b72", fontsize=8, ha="right")

# — 6. Steps per episode —
steps_roll = np.convolve(metrics["ep_steps"], np.ones(w)/w, mode='valid')
ax6 = ax_at(1, 2)
ax6.plot(ep, metrics["ep_steps"],
         color="#e3b341", lw=0.5, alpha=0.3, label="Steps/ep")
ax6.plot(ep_r, steps_roll,
         color="#e3b341", lw=1.8, label=f"Rolling avg (w={w})")
ax6.set_title("Steps per Episode")
ax6.set_xlabel("Episode")
ax6.set_ylabel("Steps")
ax6.legend(fontsize=8, facecolor=_BG, labelcolor="white",
           edgcolor="#333", framealpha=0.6)

plt.savefig(save_path, dpi=130, facecolor=_BG, bbox_inches="tight")
plt.show()
print(f"[Plot] Saved -> {save_path}")

# — Summary Table —
def print_summary_table(metrics):
    w = metrics["window"]
    rows = [
        ("Total Episodes", metrics["n_episodes"]),
        ("Total Environment Steps", f"{metrics['total_steps']:,}"),
        ("Total Successes", metrics["total_successes"]),
        ("Total Collisions", metrics["total_collisions"]),
        (f"Final Avg Reward (last {w} ep)", f"{metrics['final_avg_reward']:+.3f}"),
        (f"Final Success Rate (last {w} ep)", f"{metrics['final_success_rate']:.1f}%"),
        ("— Total Reward —", "_____"),
        (" Sum of all episode rewards", f"{metrics['total_reward_sum']:, .2f}"),
        (" Average reward per episode", f"{metrics['avg_total_reward']:+.3f}"),
        (" Best single episode reward", f"{metrics['max_episode_reward']:+.3f}"),
        (" Worst single episode reward", f"{metrics['min_episode_reward']:+.3f}"),
        ("— Convergence —", "_____"),
        (" Convergence Episode", f"ep {metrics['convergence_ep']}"),
        (" Convergence (cumul steps)", f"{metrics['convergence_steps']:,}"),
        (" Convergence Rate", f"{metrics['convergence_rate_pct']:.2f}% ")
    ]

```

```

        (" Training Remaining After Conv.",
        ("— Sample Efficiency —",
        (" DQN (this run)",
        f"(after {metrics['convergence_rate_pct']:.1f}% of training)",
        f"{metrics['post_convergence_pct']:.2f}%)",
        "_____"),
        f"{metrics['sample_eff_steps']:,} steps"),
    ]
    print(f"\n{'-'*62}")
    print(f"{' DQN Performance Summary':^62}")
    print(f"{'-'*62}")
    for k, v in rows:
        print(f"    {k:<46} {v}")
    print(f"{'-'*62}\n")

# — Save Artefacts —————
def save_artefacts(metrics, agent, model_path="dqn_model.zip", metrics_path="dqn_metrics.json"):
    """
    Save the trained DQN model weights (zipped .pt file) and
    training metrics as a JSON file.

    Files saved
    -----
    dqn_model.zip    - contains dqn_model.pt (online network state dict)
    dqn_metrics.json - all scalar/list metrics from the training run

    To reload the model later:
        model = DQNNetwork(state_dim=7, action_dim=5)
        with zipfile.ZipFile("dqn_model.zip") as zf:
            zf.extract("dqn_model.pt")
            model.load_state_dict(torch.load("dqn_model.pt"))
            model.eval()
    """
    # — 1. Save model weights (.pt) then zip —————
    pt_path = "dqn_model.pt"
    torch.save(agent.online.state_dict(), pt_path)
    with zipfile.ZipFile(model_path, "w", zipfile.ZIP_DEFLATED) as zf:
        zf.write(pt_path, arcname="dqn_model.pt")
    os.remove(pt_path)      # clean up the unzipped .pt
    print(f"[Save] Model weights -> {model_path}")

    # — 2. Serialise metrics to JSON —————
    # numpy arrays and booleans aren't JSON-serialisable by default
    def _serialise(v):
        if isinstance(v, np.ndarray):
            return v.tolist()
        if isinstance(v, (np.integer,)):
            return int(v)
        if isinstance(v, (np.floating,)):
            return float(v)
        if isinstance(v, list) and v and isinstance(v[0], (np.bool_,)):
            return [bool(x) for x in v]
        return v

    clean = {k: _serialise(v) for k, v in metrics.items()}
    with open(metrics_path, "w") as f:
        json.dump(clean, f, indent=2)
    print(f"[Save] Training metrics -> {metrics_path}")

# — MAIN —————
metrics, agent = train_dqn(
    n_episodes = 2_000,
    max_steps  = 500,
    n_people   = 3,
    n_obstacles = 4,
    seed       = 42,
    print_every = 50,
)

print_summary_table(metrics)
plot_metrics(metrics, save_path="dqn_metrics.png")
save_artefacts(metrics, agent)

```

```

Requirement already satisfied: gymnasium in /usr/local/lib/python3.12/dist-packages (1.2.3)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from gymnasium) (0.0.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)

```

```
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.13.3)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
[DQN] Device: cpu
```

```
=====
DQN Training | 2000 episodes | seed=42 | device=cpu
=====
```

Ep	1/2000	AvgR(w100)=	-224.18	TotR=	-224.18	SR=	0.0%	CR=	100.0%	eps=	0.9900	CumulSteps=	287
Ep	50/2000	AvgR(w100)=	-386.46	TotR=	-520.18	SR=	28.0%	CR=	62.0%	eps=	0.6050	CumulSteps=	9,810
Ep	100/2000	AvgR(w100)=	-220.91	TotR=	-211.99	SR=	38.0%	CR=	55.0%	eps=	0.3660	CumulSteps=	19,477
Ep	150/2000	AvgR(w100)=	-28.48	TotR=	-19.31	SR=	41.0%	CR=	57.0%	eps=	0.2215	CumulSteps=	27,107
Ep	200/2000	AvgR(w100)=	10.31	TotR=	-20.19	SR=	47.0%	CR=	53.0%	eps=	0.1340	CumulSteps=	37,667
Ep	250/2000	AvgR(w100)=	23.28	TotR=	177.18	SR=	59.0%	CR=	41.0%	eps=	0.0811	CumulSteps=	47,529
Ep	300/2000	AvgR(w100)=	32.10	TotR=	22.85	SR=	55.0%	CR=	45.0%	eps=	0.0500	CumulSteps=	57,246
Ep	350/2000	AvgR(w100)=	20.22	TotR=	89.38	SR=	43.0%	CR=	57.0%	eps=	0.0500	CumulSteps=	65,102
Ep	400/2000	AvgR(w100)=	18.31	TotR=	130.32	SR=	42.0%	CR=	58.0%	eps=	0.0500	CumulSteps=	75,004
Ep	450/2000	AvgR(w100)=	32.18	TotR=	4.79	SR=	49.0%	CR=	51.0%	eps=	0.0500	CumulSteps=	84,963
Ep	500/2000	AvgR(w100)=	29.48	TotR=	-111.71	SR=	47.0%	CR=	53.0%	eps=	0.0500	CumulSteps=	94,323
Ep	550/2000	AvgR(w100)=	35.71	TotR=	120.02	SR=	48.0%	CR=	52.0%	eps=	0.0500	CumulSteps=	104,162
Ep	600/2000	AvgR(w100)=	45.90	TotR=	114.61	SR=	54.0%	CR=	46.0%	eps=	0.0500	CumulSteps=	115,378
Ep	650/2000	AvgR(w100)=	65.46	TotR=	-2.53	SR=	62.0%	CR=	38.0%	eps=	0.0500	CumulSteps=	125,931
Ep	700/2000	AvgR(w100)=	69.85	TotR=	81.15	SR=	57.0%	CR=	43.0%	eps=	0.0500	CumulSteps=	135,366
Ep	750/2000	AvgR(w100)=	54.26	TotR=	181.26	SR=	49.0%	CR=	51.0%	eps=	0.0500	CumulSteps=	144,096
Ep	800/2000	AvgR(w100)=	40.54	TotR=	172.59	SR=	56.0%	CR=	44.0%	eps=	0.0500	CumulSteps=	154,470
Ep	850/2000	AvgR(w100)=	41.20	TotR=	-130.58	SR=	63.0%	CR=	37.0%	eps=	0.0500	CumulSteps=	165,842
Ep	900/2000	AvgR(w100)=	59.82	TotR=	71.87	SR=	65.0%	CR=	35.0%	eps=	0.0500	CumulSteps=	177,229
Ep	950/2000	AvgR(w100)=	66.72	TotR=	226.86	SR=	64.0%	CR=	36.0%	eps=	0.0500	CumulSteps=	188,533
Ep	1000/2000	AvgR(w100)=	69.04	TotR=	197.35	SR=	61.0%	CR=	39.0%	eps=	0.0500	CumulSteps=	199,533
Ep	1050/2000	AvgR(w100)=	94.34	TotR=	77.06	SR=	61.0%	CR=	39.0%	eps=	0.0500	CumulSteps=	212,579
Ep	1100/2000	AvgR(w100)=	102.08	TotR=	148.45	SR=	72.0%	CR=	28.0%	eps=	0.0500	CumulSteps=	226,023
Ep	1150/2000	AvgR(w100)=	94.44	TotR=	-8.36	SR=	76.0%	CR=	24.0%	eps=	0.0500	CumulSteps=	238,699
Ep	1200/2000	AvgR(w100)=	99.74	TotR=	117.91	SR=	65.0%	CR=	35.0%	eps=	0.0500	CumulSteps=	250,486
Ep	1250/2000	AvgR(w100)=	101.89	TotR=	-88.22	SR=	66.0%	CR=	34.0%	eps=	0.0500	CumulSteps=	263,853
Ep	1300/2000	AvgR(w100)=	87.43	TotR=	171.54	SR=	73.0%	CR=	27.0%	eps=	0.0500	CumulSteps=	276,747